

Numerical Implementation of a Stochastic Control Solver for Constrained Optimal Execution and Energy Storage

Contents

1	Introduction	2
2	C++ Library	3
2.1	Components	3
2.2	Optimization Alternatives	5
2.2.1	OpenMP Parallelization Strategy	6
2.2.2	Eigen with BLAS/LAPACK	7
2.3	PyBindings	8
2.4	CMakeLists.txt	9
2.5	Sad Note	10
3	Repository	12
3.1	Structure	12
3.2	Tests	14
3.3	Functionalities	16
3.4	Branches	18
4	Conclusion	22

1 Introduction

Stochastic optimal control problems with constraints play a central role in modern applications of quantitative finance and energy management. A prominent example is the model of Abi Jaber, De Carvalho, and Pham [1], which analyzes optimal trade execution and battery energy storage under transient market impact. In this setting, the agent seeks to determine admissible trading or charging–discharging strategies that maximize expected utility while respecting inventory constraints, trading rate limits, and liquidation requirements.

The dynamics of the system are influenced by both market frictions and stochastic fluctuations in prices, which are modeled through an Ornstein–Uhlenbeck process. The mathematical formulation of the constrained optimization problem leads to a stochastic Fredholm integral equation. Solving this equation requires combining tools from convex analysis, stochastic processes, and numerical methods. The algorithmic approach proposed in section 3, [1] is based on the stochastic Uzawa algorithm, a primal–dual scheme that alternates between updating control variables and dual multipliers associated with the constraints. Conditional expectations that arise in the dual update are approximated by least-squares Monte Carlo regression, while the underlying Fredholm equation is discretized by a Nyström scheme. This results in a computationally intensive algorithm with multiple nested components: Monte Carlo simulation, regression over polynomial bases, integral equation discretization, and iterative primal–dual updates.

The goal of the present project was not to revisit the theoretical aspects of the model, but rather to build a working numerical implementation of the algorithm and to organize it in a robust and reproducible software repository. The implementation was carried out in C++, chosen for its performance and control over memory management, and integrated with Python via `pybind11` bindings to allow analysis, visualization, and interaction within the scientific Python ecosystem. The solver was designed in a modular fashion, with separate components for simulation, regression, constraint handling, and optimization, all coordinated by a high-level interface.

Given the complexity of the algorithm, optimization of computational performance was a central concern. Two different strategies were explored: explicit loop-level parallelization using OpenMP, and library-level optimization through Eigen combined with external BLAS and LAPACK libraries. These approaches fight to coexist, and for this reason the repository maintains two distinct branches, each implementing one optimization strategy. This design facilitates clean implementation and provides a framework for performance comparison.

The repository was also structured to support reproducibility and ease of experimentation. It includes automated scripts for environment setup, test execution, result collection, and visualization. Tests were implemented at multiple levels: C++ unit testing for algorithm validation, Python interface testing for bindings verification, and an interactive application for end-user experimentation with different parameter configurations. This multi-tier testing approach ensures both correctness and usability across different environments.

The remainder of the report is organized as follows. Section 2 presents the C++ library, its components, optimization alternatives, Python bindings, and build system. Section 3 describes the repository in detail, including its structure, environment setup, testing framework, automation scripts, and branch organization. Finally, Section 4 provides concluding remarks and reports performance comparisons between the OpenMP-parallel and Eigen-optimal implementations.

2 C++ Library

2.1 Components

This section provides a detailed overview of the header files located in the `include/` directory. Each file describes a specific functionality and interacts with others during the simulation and optimization pipeline. The relative implementations of each file are collected in the `src/` directory.

CommonTypes.hpp Defines type aliases used consistently throughout the codebase:

- `Matrix = Eigen::MatrixXd`, `Vector = Eigen::VectorXd`
- `Ref = Eigen::Ref<Matrix>`, `MatVec = std::vector<Matrix>`
- `par_for_type = std::ptrdiff_t`, used in OpenMP parallel loops

Includes `omp.h` for OpenMP support. Centralizes type declarations to ensure consistent usage across all modules.

Constraints.hpp Handles inventory and trading rate constraints:

- Parameters include initial inventory `X0`, bounds on trading rate (`u_min`, `u_max`), and inventory (`X_u_min`, `X_u_max`)
- Supports terminal liquidation (forces final inventory to zero)
- Includes stop-trading constraint when price falls below a specified threshold

Main method:

- `compute_constraints(const Matrix& price)`: returns a `MatVec` containing the four constraint matrices computed based on a given price path

Interface.hpp Implements a high-level interface for accessing the stochastic Uzawa solver:

- Handles parameter parsing and validation, error messages via `ParamError`
- Provides a simplified API for running simulations and retrieving results
- Main methods:
 - `read_par()`, `update_price_par()`: parse parameters
 - `simulate_price()`: generate Ornstein–Uhlenbeck price path
 - `solve()`: execute the stochastic Uzawa algorithm
 - `get_price()`, `get_u()`, `get_X()`, `get_lambda1-4()`, etc.: access computed quantities

Kernel.hpp Implements kernel functions for the Volterra integral operators that model transient price impact:

- Base class: `Kernel` (abstract interface)
- Two concrete implementations:
 - `ExpKernel`: $K(t, s) = c \cdot e^{-\rho(t-s)}$, for $t \geq s$
 - `FracKernel`: $K(t, s) = c \cdot (t - s)^{-\alpha}$, for $t > s$

- Main methods:
 - `fun(t,s)`: evaluate $K(t,s)$
 - `s_integral(t, a, b)` and `t_integral(s, a, b)`: compute definite integrals
 - `check_integral_params<IntegralType>()`: ensures parameter validity for quadrature

LSMCR.hpp Implements Least Squares Monte Carlo Regression to estimate conditional expectations:

- Regression 1: $\mathbb{E}_i \left[\sum_{l=i+1}^N (\lambda_{3,l} - \lambda_{4,l}) \right]$
- Regression 2: $\mathbb{E}_i \left[\lambda_{1,j} - \lambda_{2,j} + \sum_{l=j+1}^N (\lambda_{3,l} - \lambda_{4,l}) \right]$
- Uses Laguerre polynomials on three state variables: α, Z_u, X_u
- Main methods:
 - `get_target_i()`, `get_target_ij()`: build targets for both regressions
 - `get_regressors_i()`, `get_regressors_ij()`: build basis functions
 - `solve_regression_i()`, `solve_regression_ij()`: solve least squares systems
 - `get_cond_exp_i()`, `get_cond_exp_ij()`: return expectation estimates

NystromScheme.hpp Applies the Nyström method to discretize the Volterra integral equations:

- Constructs matrix operators:
 - `construct_time_integral()`, `construct_L_and_U()`, `construct_inv_Dt()`
 - `construct_B_and_a_operator()`: builds system matrices
 - `update_state_variables()`: updates control, inventory and price impact paths

OUSimulator.hpp Simulates Ornstein–Uhlenbeck process with periodic mean:

- Dynamics: $dI_t = (A(t) - kI_t) dt + \psi dW_t$, where $A(t) = \theta \sin(\omega t + \phi)$
- Main methods:
 - `generateBrownianMotion()`, `generateOrnsteinUhlenbeck()`
 - `compute_Pt()`, `compute_price()`, `compute_alpha_R()`
- Key parameters: $\sigma, S_0, I_0, \theta, \omega, \phi, k, \psi$, plus random seeds

Parameters.hpp Centralized parameter management:

- Structures:
 - `NumericSchemeParams`: $T, N, D, M, \varepsilon, \delta, \beta, d1, d2$
 - `OUParams`: $\sigma, S_0, I_0, \theta, \omega, \phi, k, \psi$
 - `ConstraintsParams`, `KernelParams`
- Methods: `read_parameters()`, `print_parameters()`, multiple getters
- Utility: `degrees_to_radians()` converts angles for OU phase shift

StochasticUzawaSolver.hpp Main implementation of the stochastic Uzawa algorithm:

- Coordinates all solver modules: `OUSimulator`, `Constraints`, `Kernel`, `NystromScheme`, `LSMCR`
- Main methods:
 - `setup_variables()`,
 - `simulate_signal()`
 - `update_parameters()`,
 - `compute_slackness()`, `gradient_update()`
 - `solve()`
- Algorithm:
 - First: gradient ascent on Lagrange multipliers λ
 - Second: conditional Expectation estimation via LSMCR regression
 - Third: Nystrom update on control u
 - Compute slackness for convergence check

Chrono.hpp Provides basic timing utilities for benchmarking execution time. Included for performance testing.

GetPot External library for parsing configuration files. Used to read simulation and solver parameters from text files.

System Architecture Overview:

1. `Parameters.hpp` → Read parameters
2. `OUSimulator.hpp` → Generate price paths
3. `Constraints.hpp` → Compute admissible regions
4. `Kernel.hpp` → Define price impact kernel
5. `NystromScheme.hpp` → Discretize integral operators
6. `LSMCR.hpp` → Estimate conditional expectations
7. `StochasticUzawaSolver.hpp` → Core of the algorithm
8. `Interface.hpp` → Expose user-facing API

2.2 Optimization Alternatives

The stochastic Uzawa solver involves heavy numerical computation, including nested Monte Carlo loops, matrix operations, and iterative optimization steps. The main computational bottlenecks are:

- Monte Carlo simulation over M sample paths
- Dense matrix operations involving $N \times N$ matrices
- Least Squares Monte Carlo Regression with polynomial basis functions

- Nyström scheme discretization for Volterra integral equations
- Iterative primal-dual optimization with convergence checking

To address these computational demands, two optimization strategies were implemented:

1. Manual parallelization using OpenMP: explicit control over parallel loops
2. Automatic optimization via Eigen with external BLAS/LAPACK: integration with optimized linear algebra backends

2.2.1 OpenMP Parallelization Strategy

Approach Overview. OpenMP directives were applied manually to performance-critical regions to provide explicit control over loop-level parallelism.

Implementation Summary. OpenMP was used in three main modules:

- LSMCR.cpp (Least Squares Monte Carlo Regression)
 - 13 parallel regions targeting regression loops
 - Parallelization techniques:
 - * `#pragma omp parallel for schedule(static)`
 - * `#pragma omp parallel for collapse(2)`
 - * `#pragma omp parallel sections`
 - Specific routines:
 - * `precompute_target_i()`, `precompute_target_ij()`: compute regression targets
 - * Laguerre polynomial basis evaluation across paths
 - * Construction of basis matrices and regression solving
- NystromScheme.cpp (Volterra Integral Solver)
 - 5 parallel regions in integration matrix construction
 - Examples:
 - * `construct_L_and_U()`: parallel integral computation over t_i
 - * `construct_inv_Dt()`: time-step-wise inverse computation of $(I + K_t + K_t^*)^{-1}$
 - * `construct_B_and_a_operator()`: matrix products parallelized row-wise
- OUSimulator.cpp (Price Simulation)
 - 6 parallel regions across simulation components
 - Parallel tasks:
 - * Brownian motion generation
 - * Euler–Maruyama simulation of OU process
 - * Price transformation and conditional expectation computation

Scheduling Strategies.

- `schedule(static)`: Used for uniform path-wise workload
- `collapse(2)`: Applied to flatten nested loops
- `parallel sections`: Used for independent regression tasks

CMake Configuration for OpenMP. OpenMP support is explicitly defined in `CMakeLists.txt`:

```
find_package(OpenMP REQUIRED)
target_link_libraries(test_cpp Eigen3::Eigen OpenMP::OpenMP_CXX)
```

Eigen parallelism is disabled to avoid thread conflicts:

```
add_compile_definitions(EIGEN_DONT_PARALLELIZE)
add_compile_definitions(EIGEN_NO_AUTOMATIC_RESIZING)
```

Compiler optimization flags:

```
set(CMAKE_CXX_FLAGS_RELEASE "-O3 -DNDEBUG")
```

2.2.2 Eigen with BLAS/LAPACK

Approach Overview. The second strategy uses Eigen's ability to delegate core linear algebra operations to external BLAS and LAPACK libraries. This allows implicit optimization via vectorized kernels and batched solvers.

Implementation Challenges. Due to library and compiler mismatches in the cluster environment, system-provided BLAS libraries were incompatible. Therefore, all dependencies were built manually:

- Toolchain: `gcc-glibc/11.2.0`
- OpenBLAS: Compiled with pthreads, installed at `$HOME/local/openblas-tc/`
- LAPACK/LAPACKE: Reference implementation compiled statically and installed in `$HOME/local/lapack-tc/`

CMake Integration. Custom detection and linking logic for external libraries:

```
# Define library paths
set(OPENBLAS_LIB "$ENV{HOME}/local/openblas-tc/lib/libopenblas.so")
set(LAPACK_INC "$ENV{HOME}/local/lapack-tc/include")
set(LAPACK_LIB "$ENV{HOME}/local/lapack-tc/lib/liblapack.a")

# Fallback to system libraries
if(NOT EXISTS "${OPENBLAS_LIB}")
    message(WARNING "OpenBLAS not found, trying system BLAS")
    find_package(BLAS)
endif()
```

Enable Eigen integration with external backends:

```
target_compile_definitions(test_cpp PRIVATE
    EIGEN_USE_BLAS
    EIGEN_USE_LAPACK
    EIGEN_USE_THREADS
    EIGEN_NO_DEBUG
    NDEBUG
)
```

Explicit linking and runtime path setup:

```
target_link_libraries(test_cpp PRIVATE
    ${LAPACK_LIB}
    ${OPENBLAS_LIB}
    Threads::Threads
    m dl
)

set_target_properties(test_cpp PROPERTIES
    BUILD_RPATH "$ENV{HOME}/local/openblas-tc/lib"
    INSTALL_RPATH "$ENV{HOME}/local/openblas-tc/lib"
)
```

Compiler Optimization Flags. Aggressive compilation flags used in both C++ and Python modules:

```
target_compile_options(test_cpp PRIVATE
    -O3
    -march=native
    -mtune=native
)
```

Explanation:

- `-O3`: High-level optimization, including loop unrolling and SIMD
- `-march=native`: Enables CPU-specific instructions (e.g., AVX, SSE)
- `-mtune=native`: Tunes scheduling and execution for host microarchitecture

Threading Behavior. Unlike OpenMP, threading is managed internally by the BLAS library:

```
setenv("OPENBLAS_NUM_THREADS", num_threads_str.c_str(), 1);
Eigen::setNbThreads(num_threads);
```

Note: When linked against external BLAS, `Eigen::setNbThreads()` is typically ineffective. Thread count should instead be set via environment variables such as `OPENBLAS_NUM_THREADS`.

2.3 PyBindings

The C++ Stochastic Uzawa solver was made accessible from Python using `pybind11`. This integration allows seamless use of the solver within the Python scientific ecosystem for analysis, sensitivity studies, and visualization.

Technical Implementation

The bindings expose two classes:

- `ParamError`: Container for parameter validation errors, with attributes for error path and error message.
- `Interface`: Main solver interface providing:
 - `read_par()`: Load parameter files with validation
 - `update_price_par()`: Update price parameters while preserving other settings
 - `simulate_price()`: Generate Ornstein–Uhlenbeck price paths
 - `solve()`: Execute the Uzawa optimization algorithm
 - Getters: `get_u()`, `get_X()`, `get_lambda1-4()`, `get_price()`, `get_time_grid()`

Matrix Integration

Matrix data is handled through `pybind11/eigen.h`, enabling zero-copy conversion between Eigen matrices and NumPy arrays. The binding policy `return_value_policy::reference_internal` ensures memory safety by maintaining references to underlying C++ objects.

Most matrices follow the convention:

$$(M, N)$$

where M is the number of Monte Carlo paths and N is the number of time steps.

Dual Build System

Note that the `CMakeLists.txt` compile and link this file depending on the value of an option. This option allows two configurations:

- `BUILD_BINDINGS=OFF`: Build a standalone C++ test executable
- `BUILD_BINDINGS=ON`: Build a Python module with `pybind11`

Both modes share identical optimization flags and linking configuration, ensuring consistent performance between the C++ and Python builds, check following section for further details.

Usage Workflow

A typical Python workflow is illustrated below:

```
from stochastic_uzawa import Interface

interface = Interface()
errors = interface.read_par("Parameters.pot")
price = interface.simulate_price()    # Shape: (M, N)
interface.solve()

u = interface.get_u()                # Optimal control
X = interface.get_X()                # Inventory trajectory
lambdas = [interface.get_lambda1(), interface.get_lambda2(),
            interface.get_lambda3(), interface.get_lambda4()]
```

This interface enables Python users to perform Monte Carlo ensemble analysis, parameter sensitivity studies, and visualization of the computed control and multiplier trajectories.

2.4 CMakeLists.txt

The `CMakeLists.txt` implements a dual-target build system controlled by the `BUILD_BINDINGS` option. This setup allows building either a standalone C++ executable or a Python extension module from the same source files.

Build Configuration

Common setup:

- C++20 standard with release optimization (`-O3 -DNDEBUG`)
- Comprehensive Eigen3 path detection
- Source file collection from the `src/` directory
- Consistent compiler flags across both build targets

Build Targets

C++ Executable (BUILD_BINDINGS=OFF, default).

Target: test_cpp (executable)
Sources: src/*.cpp + tests/cpp/test_cpp.cpp
Output: tests/cpp/build/test_cpp
Purpose: Testing only | final production interface is Python
Usage: ./tests/cpp/build/test_cpp -i data/Parameters.pot -o output_folder_name

Python Module (BUILD_BINDINGS=ON).

Target: stochastic_uzawa (Python module)
Sources: src/*.cpp + bindings/python_bindings.cpp
Output: stochastic_uzawa.so (dynamic library)
Purpose: Production interface
Usage: import stochastic_uzawa

Key Differences

Aspect	C++ Executable	Python Module
Build Type	Direct executable	Dynamic library via pybind11
Entry Point	main() function	PYBIND11_MODULE macro
Dependencies	System libraries only	pybind11 + Python headers
Distribution	Self-contained binary	Python package component
Interface	Command-line arguments	Python API

Technical Implementation

Python Module. The Python module is compiled into a shared library (.so) that is dynamically loaded at runtime upon import. This provides a lightweight workflow: modifications to the C++ code require recompilation of the library only, without touching dependent Python scripts. Installation and packaging are managed via `pyproject.toml`, which specifies build dependencies and ensures integration with Python virtual environments, check dedicated section.

C++ Executable. For testing purposes, a single binary is built by directly compiling the source files without creating intermediate static libraries. Since this build serves exclusively for debugging and validation, the absence of a static archive simplifies the process.

Library Detection. The build system automatically detects `pybind11` using CMake's `find_package` mechanism. If unavailable, it falls back to detection through Python module import. Both build targets link against the same numerical libraries, ensuring performance parity between the C++ and Python interfaces.

2.5 Sad Note

This section is somewhat unusual for a scientific report but it is important for explaining the design choices made in the remainder of the project.

The target algorithm was extremely complex given the limited time available. The project was successful in terms of understanding the algorithmic logic and developing a working implementation of its components. However, a numerical error remains that prevents the convergence predicted in the original paper.

For this reason, rather than focusing on presenting results with refined plots and extensive tests of algorithmic accuracy, I directed my efforts toward other important aspects of the project: ensuring smooth interaction between the different source files, designing shell scripts (.sh) to

automate experiment setup and output management, and improving performance through two distinct optimization strategies.

3 Repository

The repository serves as the backbone of the project, collecting source code, build configuration, test scripts, and automation tools in a coherent structure. Beyond providing an implementation of the stochastic Uzawa algorithm, the repository was designed to support reproducibility, testing, and performance evaluation.

The following subsections provide a detailed description of its components:

- **Structure:** overview of the directory tree and organization of project files.
- **Environment:** description of the requirements and setup needed to run simulations and tests.
- **Tests:** explanation of the available tests and their purposes.
- **Functionalities:** description of the shell scripts (`.sh`) that automate the workflow, from environment setup to the execution of full experiments.
- **Branches:** motivation for maintaining two separate branches, each dedicated to a different optimization strategy.

3.1 Structure

This subsection provides an overview of the repository organization and the role of each folder and key file. The structure was designed to balance numerical performance, usability through Python integration and automated testing.

The repository follows a separation of concerns:

- Core algorithms in C++ for performance (`include/`, `src/`)
- Python integration for usability and analysis (`bindings/`, `pyproject.toml`)
- Automation through scripts and build system (`scripts/`, `CMakeLists.txt`)
- Testing and application interfaces (`tests/`, `apps/`)
- Documentation and reproducibility (`doc/`, various README files)

Core Implementation Folders

`include/` and `src/` C++ header-source files implementing the mathematical components of the stochastic Uzawa solver. See dedicated section for further details, 2.1.

`bindings/` Python bindings implemented with `pybind11`, exposing the C++ functionality to Python. Provides automatic conversion between Eigen matrices and NumPy arrays. See dedicated section for further details, 2.3.

Configuration and Data

`data/` Contains parameter configuration files:

- `Parameters_template.pot`: read-only template with default values and documentation. Permissions can be enforced with:
`chmod 444 /home/jammy/tesi/stochastic_uzawa/data/Parameters_template.pot`
- `Parameters.pot`: working file that can be modified for simulations

Users can duplicate the template to create custom parameter files for different market scenarios or algorithmic settings.

pyproject.toml Python package configuration file following PEP 518 standards. Defines the build system (**scikit-build-core** with **pybind11**), project metadata, dependencies (mandatory **numpy**, optional **matplotlib**), and build configuration (CMake arguments with **-DBUILD_BINDINGS=ON**). Requires Python ≥ 3.8 and allows installation via **pip install ..**

CMakeLists.txt Build configuration for compiling C++ code. Implements a dual-target system to generate either a C++ executable or a Python extension module. A detailed description is provided in section 2.4.

Automation and Testing

scripts/ Shell scripts for workflow automation. Includes environment setup, virtual environment management, repository cleaning, build automation, and execution of experiments from parameter configuration to results. Documented in section 3.3.

tests/ Validation and testing framework. Contains:

- **cpp/**: unit tests and validation for C++ modules
- **python/**: tests for Python bindings and integration

Both subfolders contain test drivers and validation scripts.

apps/ Interactive application for end-users. Provides a production-ready interface for running simulations with parameter management and organized outputs.

For further details about tests go to dedicated section 3.2.

Utilities and Output

tools/ Utility scripts for visualization and analysis. The main component is **plotter.py**, which can:

- parse serialized text outputs from C++ tests and generate plots,
- directly consume NumPy arrays from Python tests,
- be used either as an imported class or as a standalone tool.

The graphical quality of plots was not prioritized, for reasons explained in section 2.5.

doc/ Documentation folder containing the project report, technical documentation, and reference papers.

outputs/ Automatically generated directory for simulation results:

- **cpp/**: results from C++ executable runs
- **python/**: results from Python module runs
- **app/**: results from the interactive application

Subfolders are organized by test name and timestamp to facilitate result management and contains plots and summary specific for the type of test.

Virtual Environment

venv/ This folder hosts the Python virtual environment used to run the project. It contains the compiled `stochastic_uzawa` package, required dependencies (`numpy`, `matplotlib`, `pybind11`), and provides an isolated runtime environment for development and testing.

The virtual environment is not tracked in the repository and must be created after cloning. Two alternatives are available:

Manual setup. Users may create the environment by running:

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install numpy==1.24.3 matplotlib pybind11 scikit-build-core
```

Automated setup. Alternatively, the same procedure can be performed by executing the dedicated shell script available in the `scripts/` folder. This approach is recommended for convenience and consistency. A detailed description of the script is provided in the section on functionalities, 3.3.

3.2 Tests

The repository includes three testing environments: C++ algorithm testing, Python bindings validation, and interactive experimentation. Together they provide low-level validation of performance and correctness, verification of the Python interface, and end-user oriented exploration of the algorithm.

C++ Algorithm Testing (/tests/cpp)

Purpose. The C++ test framework evaluates algorithm performance and supports debugging. It provides low-level access to core implementations for detailed analysis and optimization.

Functionality.

- Performance testing using the `chrono` library for execution time measurement
- Algorithm validation by checking iterative solver updates across multiple runs
- Debugging support through detailed output of algorithm states
- Cross-branch comparison, since outputs differ between the `openmp-parallel` and `eigen-optimal` branches, see dedicated section 3.4

Manual Build. The build process depends on the active branch because each branch maintains a separate `CMakeLists.txt`. Detailed build commands are given in the branches section, 3.4.

Manual Execution. C++ tests can be run manually as follows:

```
cd /home/jammy/tesi/stochastic_uzawa/tests/cpp
./test_cpp -i <parameter_file> -o <output_folder>
```

where `<parameter_file>` is a path to a parameter file (e.g. `data/Parameters.pot`) and `<output_folder>` is the name of a results folder automatically created in `outputs/cpp/`.

Automated Execution. A dedicated shell script is available for automated execution. Details are provided in the section on automation scripts, 3.3.

Python Bindings Testing (/tests/python)

Purpose. The Python test framework validates the bindings created with `pybind11`, ensuring that the Python C++ solver can be used correctly, and it is the most direct test to verify that the output is actually correct because the outputs are lighter.

Functionality.

- Validation of the Python–C++ bindings and data transfer
- Verification that algorithm outputs through Python match the C++ results
- Testing of NumPy integration and correct matrix handling
- Verification of all exposed API functions and classes
- Handy for correctness check

Manual Execution. Python tests can be run manually as follows:

```
cd /home/jammy/tesi/stochastic_uzawa/tests/python
python test_bindings.py <parameter_file> <output_folder>
```

where `<parameter_file>` is a path to a parameter file (e.g. `data/Parameters.pot`) and `<output_folder>` is the name of a results folder automatically created in `outputs/python/`.

Automated Execution. A dedicated shell script is available for automated execution. Details are provided in the automation scripts section, 3.3.

Interactive Experimentation (apps/)

Purpose. The interactive application provides an end-user interface for exploring the stochastic Uzawa solver with real-time parameter adjustment and visualization.

Application Flow.

1. Parameter configuration phase:
 - Read parameters from configuration files
 - Validate consistency and enforce mathematical constraints
 - Loop until all parameter values are valid
2. Price simulation visualization:
 - Generate Ornstein–Uhlenbeck simulations
 - Display interactive plots of price trajectories (saved under `outputs/app/experiment_name/price.png`)
 - Allow regeneration with different parameters until user’s happiness
3. Problem solving phase:
 - Run the stochastic Uzawa optimization algorithm
 - Produce solution outputs including controls, multipliers, and convergence metrics
 - Display final visualizations of results and performance

Manual Execution. The interactive experiment can be run manually as follows:

```
cd /home/jammy/tesi/stochastic_uzawa/apps
python interactive_experiment.py <parameter_file> <output_folder>
```

where `<parameter_file>` is a path to a parameter file (e.g. `data/Parameters.pot`) and `<output_folder>` is the name of a results folder automatically created in `outputs/app/`.

Automated Execution. A shell script is available for automated execution. Its usage is described in the automation scripts section, 3.3.

3.3 Functionalities

The repository provides a set of shell scripts that automate environment setup, compilation, execution, and result visualization. This subsection documents each script, its usage, parameters, and file operations. Scripts are organized under `scripts/` and `tests/`.

`scripts/cleaner.sh`.

Repository cleaning and maintenance script. Removes build artifacts, output folders, and temporary files.

Usage.

```
./cleaner.sh [option|path]
```

Options.

- `cpp`: remove `outputs/cpp`
- `python`: remove `outputs/python`
- `build_cpp`: remove `tests/cpp/build`
- `build_py`: remove `tools/__pycache__`
- `params`: remove all files in `data/` except `Parameters.pot` and `README.md`
- `venv`: remove virtual environment
- `su_lib`: uninstall `stochastic_uzawa` library from `venv`
- `outputs`: remove entire `outputs` directory
- `all`: clean outputs, build, and cache while preserving `venv` and library
- `reset`: full reset including `venv` and library
- `[path]`: remove specific path
- `help`: show usage information

Scope of Action. The script operates on `outputs/`, `tests/cpp/build/`, and `tools/__pycache__`. For uninstalling the library, it activates virtual environment running `source venv/bin/activate`.

scripts/create_venv.sh.

Creates a Python virtual environment with required dependencies.

Usage.

```
./create_venv.sh [soft|hard]
```

Options.

- **soft**: skip creation if **venv** already exists
- **hard**: remove and recreate existing **venv**

Scope of Action. The script creates **venv/**, installs **numpy==1.24.3**, **matplotlib**, **pybind11**, and **scikit-build-core**, and activates the environment for package installation.

scripts/python_setup.sh.

Performs complete Python setup, including virtual environment creation and installation of the **stochastic_uzawa** package.

Usage.

```
./python_setup.sh [soft|hard|lib]
```

Options.

- **soft**: skip if **venv** and library already installed
- **hard**: recreate **venv** and reinstall library
- **lib**: reinstall library while preserving existing **venv**

Scope of Action. The script calls **create_venv.sh**, activates **venv**, installs the package with **pip install -e .**, uses **pyproject.toml** for package configuration, and verifies import with **python -c "import stochastic_uzawa"**.

apps/run_app.sh.

Automates the full workflow of running the interactive application.

Usage.

```
./run_app.sh <soft|hard|lib> <parameter_file> [output_folder_name]
```

Parameters.

- **<soft|hard|lib>**: environment setup option (forwarded to **python_setup.sh**)
- **<parameter_file>**: path to parameter file
- **[output_folder_name]**: optional name (default: timestamped)

Scope of Action. The script sets up the environment, activates **venv**, executes **apps/interactive_experiment.py**, and stores results under **outputs/app/output_folder_name**.

tests/cpp/test_cpp.sh.

Automates C++ testing, including build, execution, and plotting.

Usage.

```
./tests/cpp/test_cpp.sh <parameter_file> [output_folder_name]
```

Scope of Action. It creates **tests/cpp/build/**, configures CMake with **-DBUILD_BINDINGS=OFF**, builds the executable, runs it, activates **venv**, and calls **tools/plotter.py** on the output. Results are stored under **outputs/cpp/output_folder_name**.

tests/python/test_bindings.sh.

Automates Python binding tests, including environment setup and execution.

Usage:

```
./tests/python/test_bindings.sh <soft|hard|lib> <parameter_file> [output_folder_name]
```

Scope of Action. It calls `python_setup.sh`, activates `venv`, executes `tests/python/test_bindings.py`, and stores results under `outputs/python/`.

Script Relationships and Dependencies

- `create_venv.sh` is called by `python_setup.sh`
- `python_setup.sh` is called by `run_app.sh` and `test_bindings.sh`
- `tools/plotter.py` is called by `test_cpp.sh`

All scripts are executed from the project root and handle absolute path resolution automatically. Parameter files are usually taken from `data/Parameters.pot`, while outputs follow the convention `outputs/<type>/<folder>/`. Timestamped folders avoid conflicts between runs. All scripts assume a Bash-compatible environment.

3.4 Branches

This subsection documents the dual-branch architecture implemented for performance optimization in the stochastic Uzawa solver. The repository maintains two main branches: one optimized with manual OpenMP parallelization, and another leveraging Eigen with external BLAS/LAPACK libraries.

Rationale for Dual-Branch Architecture

Two separate branches were implemented rather than a single configurable system. The build system was already complex, and introducing conditional logic to support mutually exclusive optimization strategies would have increased complexity without practical benefit.

The two approaches are mutually exclusive, as both rely on multithreading and can cause conflicts or overhead if combined. Finally the two approaches show many implementation differences:

- OpenMP parallelization requires explicit pragmas and manual control of thread scheduling, which means acting on source code.
- Eigen with BLAS/LAPACK delegates threading to external libraries, which means acting on build code (`CMakeLists.txt`).

It follows that maintaining separate branches ensures clean implementation and facilitates performance comparison.

Given that, the workflow was the following: after implementing manual OpenMP parallelization with measurable gains, the Eigen+BLAS/LAPACK strategy was developed in a dedicated branch. This allowed exploration of both approaches without compromises or compatibility overhead.

Implementation Complexity Comparison

OpenMP branch (openmp-parallel).

- Code modification: extensive, with `#pragma omp` directives inserted throughout the code-base.
- Complexity: arises from manual thread management and load distribution.
- Threading control: explicit, via environment variables and pragma schedules.
- Build complexity: moderate, requiring OpenMP runtime linking.

Eigen optimization branch (eigen-optimal).

- Code modification: minimal.
- Complexity: primarily in build system configuration and external library linking.
- Threading control: delegated to BLAS/LAPACK implementations.
- Build complexity: high, due to manual compilation of custom libraries and linking requirements.

Manual Build Commands

OpenMP branch. Build command:

```
cmake <PROJECT_ROOT> -DBUILD_BINDINGS=<ON|OFF>
make
```

CMake configuration (excerpt):

```
cmake_minimum_required(VERSION 3.16)
project(StochasticUzawa CXX)

option(BUILD_BINDINGS "Build Python bindings" OFF)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_FLAGS_RELEASE "-O3 -DNDEBUG")

find_package(Eigen3 QUIET)
find_package(OpenMP REQUIRED)

add_compile_definitions(EIGEN_DONT_PARALLELIZE)
add_compile_definitions(EIGEN_NO_AUTOMATIC_RESIZING)
```

Key options:

- `BUILD_BINDINGS`: toggles Python module vs C++ executable
- `EIGEN_DONT_PARALLELIZE`: disables Eigen threading to avoid conflict with OpenMP
- `EIGEN_NO_AUTOMATIC_RESIZING`: reduces overhead from dynamic resizing
- Linking: both targets (`test.cpp` and Python module) link against `Eigen3::Eigen` and `OpenMP::OpenMP_CXX`

Eigen+BLAS/LAPACK branch. Build command:

```
cmake <PROJECT_ROOT> -DBUILD_BINDINGS=<ON|OFF>
make
```

CMake configuration (excerpt):

```
cmake_minimum_required(VERSION 3.16)
project(StochasticUzawa VERSION 0.1.0 LANGUAGES CXX)

option(BUILD_BINDINGS "Build Python bindings" OFF)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_FLAGS_RELEASE "-O3 -DNDEBUG")

target_compile_options(<target> PRIVATE -O3 -march=native -mtune=native)

target_compile_definitions(<target> PRIVATE
    EIGEN_USE_BLAS
    EIGEN_USE_LAPACKE
    EIGEN_USE_THREADS
    EIGEN_NO_DEBUG
    NDEBUG
)

set(OPENBLAS_LIB "$ENV{HOME}/local/openblas-tc/lib/libopenblas.so")
set(LAPACKE_LIB "$ENV{HOME}/local/lapack-tc/lib/liblapacke.a")
set(LAPACKE_INC "$ENV{HOME}/local/lapack-tc/include")
```

Key options:

- CPU-specific optimization with `-march=native` and `-mtune=native`
- Delegation of linear algebra to external BLAS/LAPACK libraries
- Explicit paths to toolchain-compiled OpenBLAS and LAPACKE

Target linking (custom libraries):

```
target_link_libraries(<target> PRIVATE
    ${LAPACKE_LIB}
    ${OPENBLAS_LIB}
    Threads::Threads
    m dl
)
```

Fallback linking (system libraries):

```
target_link_libraries(<target> PRIVATE
    ${LAPACK_LIBRARIES}
    ${BLAS_LIBRARIES}
    Threads::Threads
)
```

Testing Methodology Differences

OpenMP branch.

- Thread scaling analysis with varying `OMP_NUM_THREADS`
- Manual control via `export OMP_NUM_THREADS=N`
- Validation of scalability and parallel efficiency

Eigen+BLAS/LAPACK branch.

- Threading managed internally by BLAS libraries
- No manual thread configuration required
- `Eigen::setNbThreads()` becomes inactive when external BLAS is enabled

Key observation: defining `EIGEN_USE_BLAS` delegates matrix operations to external BLAS, transferring control of threading to the library implementation.

Branch Selection Strategy

The production branch is selected empirically based on performance testing. Both branches are fully functional and allow switching between optimization strategies without loss of features. The choice depends on observed scalability and runtime efficiency on the target hardware.

4 Conclusion

The project aimed not only to implement the stochastic Uzawa algorithm but also to evaluate the computational performance of two distinct optimization strategies: manual OpenMP parallelization and Eigen-based optimization with external BLAS/LAPACK libraries. This section summarizes the benchmark results and provides a comparative analysis.

Experimental Setup

All experiments were run with identical model parameters, corresponding to a simulation horizon $T = 1$, discretized into $N = 200$ time steps, and $M = 1000$ Monte Carlo paths. The Ornstein–Uhlenbeck parameters were set to $\sigma = 0.1$, $S_0 = 100$, $I_0 = -2$, $\theta = -20$, $\omega = 0$, $\phi = 270^\circ$, $k = 1.0$, and $\psi = 4.0$, with random seeds fixed at zero. Constraints included an initial inventory $X_0 = 5$, trading rate bounds $[-1000, 1000]$, and mandatory terminal liquidation. The transient impact kernel was exponential with parameters $(c, \rho) = (1.0, 0.5)$. These settings yield a price matrix of dimension 1000×200 , which is representative of a non-trivial yet tractable problem size.

Two sets of experiments were performed: one on the **eigen-optimal** branch (BLAS/LAPACK integration, single-threaded OpenBLAS), and one on the **openmp-parallel** branch (manual loop parallelization, Eigen internal threading disabled).

Eigen + BLAS/LAPACK Results

The Eigen-optimal branch employed automatic BLAS/LAPACK backends with external threading control delegated to OpenBLAS. Five independent runs were executed. Results are summarized in Table 1.

Metric	Average [μ s]	Min	Max	Std. Dev. [%]
Signal	319,194	282,016	441,099	19.18
Solver	2.46×10^6	2.09×10^6	3.16×10^6	16.27

Table 1: Performance statistics for the Eigen+BLAS/LAPACK implementation (5 runs).

The Eigen branch showed relatively high variability across runs, with solver times ranging between 2.1 and 3.2 million microseconds. This is consistent with the overhead of delegating threading and memory allocation to external libraries, where fine-grained control is limited.

OpenMP Parallel Results

The OpenMP branch exploited manual loop parallelization with explicit control of thread count. Each configuration was run three times, and averages are reported in Table 2.

Threads	Avg. Signal [μ s]	Avg. Solver [μ s]	Speedup	Std. Dev. [%]
1	312,329	2.01×10^6	1.00	0.85
2	235,065	1.20×10^6	1.68	1.59
4	209,427	8.30×10^5	2.42	0.95
6	208,474	8.50×10^5	2.36	7.01
8	215,337	8.44×10^5	2.38	3.33
12	226,742	8.57×10^5	2.34	3.05

Table 2: Performance statistics for the OpenMP-parallel implementation. Each entry is the average of three runs.

The OpenMP implementation showed consistent speedup up to four threads, achieving a maximum improvement of approximately $2.4\times$ over the single-thread baseline. Beyond four threads, performance plateaued and slightly degraded due to scheduling overhead and memory contention, which are common in Monte Carlo and regression workloads.

Comparison and Discussion

The comparison highlights distinct characteristics of the two optimization strategies:

- The Eigen-optimal branch is simpler to maintain, requires minimal code modification, and benefits from highly tuned linear algebra kernels. However, performance showed significant variability and did not reach the level of acceleration provided by explicit OpenMP parallelization.
- The OpenMP-parallel branch required extensive manual code modification but provided control over parallel execution. This resulted in reproducible and stable speedups, with the solver time reduced by more than a factor of two when using four threads.

Overall, the OpenMP branch proved superior for the tested configuration, both in terms of raw performance and consistency. The Eigen+BLAS/LAPACK approach remains attractive for environments where manual parallelization is impractical or where external libraries can be better tuned. The dual-branch architecture of the repository enables both approaches to coexist, providing flexibility and a framework for future exploration on different hardware architectures. Although convergence was not achieved due to suspected discretization errors (see Section 2.5), the performance analysis demonstrates the effectiveness of the repository design and optimization strategies.

References

- [1] E. Abi Jaber, N. De Carvalho, and H. Pham, *Trading with Propagators and Constraints: Applications to Optimal Execution and Battery Storage*, arXiv:2409.12098v1 [math.OC], 18 Sep 2024.